

Indraprastha Institute of Information Technology,  
Delhi

DEPARTMENT OF COMPUTER SCIENCE

# **QUICK COMMERCE DEMAND & SUPPLY DATABASE SYSTEM**

Course: Fundamentals of Database Management Systems

Business Requirements Document (BRD)

**Submitted By (Group Members):**

**Palak Maurya (2024400)**

**Akshat Gupta (2024057)**

**Anand Raj (2024068)**

**Project Group Number: 78**

**Tutorial Section: Tutorial 6**

**Teaching Assistants: Nidhi Jha, Mann Khatri**

# 1. Executive Summary

The Quick Commerce Demand & Supply Database System is designed to model the backend operations of an instant delivery platform (similar to Blinkit, Zepto, or Instamart). The primary goal is to facilitate ultra-fast delivery (10–20 minutes) by efficiently managing inventory across distributed dark stores (warehouses), matching customer demand with real-time supply, and ensuring data consistency during high-volume transactions.

This document outlines the business, functional, and data requirements necessary to design a robust database schema that supports order fulfillment, stock replenishment, and operational analytics.

## 2. Problem Statement

Traditional e-commerce models struggle with the unique constraints of quick commerce:

- **Distributed Inventory Fragmentation:** Stock is distributed across hundreds of micro-warehouses rather than central hubs.
- **Latency Sensitivity:** Inventory checks must be instantaneous to prevent overselling during peak hours.
- **Perishability:** Managing expiration dates (batches) for groceries requires precise tracking First Expiry, First Out (FEFO)
- **Data Integrity:** High concurrency (thousands of simultaneous orders) risks race conditions where stock counts become inconsistent.

These challenges cannot be reliably addressed using file-based systems and instead require a transactional relational database.

To address these issues, the system relies on the ACID properties of a relational database. The system enforces Atomicity through transactions (all-or-nothing stock updates), Consistency via constraints (foreign keys, check constraints), Isolation through MySQL isolation levels (e.g., Repeatable Read) to handle concurrent orders, and Durability to ensure committed data survives failures. MySQL’s InnoDB engine is utilized to guarantee these properties.

## 3. Objectives

- **Real-time Inventory Accuracy:** Ensure stock levels reflect physical reality instantly after every transaction.
- **Demand-Supply Matching:** Automatically route orders to the nearest serviceable warehouse with sufficient stock.
- **Auditability:** Maintain a granular ledger of all stock movements (replenishment, sales, damage, returns).
- **Scalability:** Design a schema that supports rapid addition of new products, categories, and warehouses.
- **Data Integrity:** Ensure data integrity using primary keys, foreign keys, and constraints.
- **Concurrency Support:** Support concurrent transactions using isolation and rollback mechanisms.

## 4. Stakeholders & User Roles

| Role             | Description & Primary Goal                                                                                               |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| Customer         | End-user of the mobile/web app. Goals: Browse catalog, check availability, place orders, track order status.             |
| Delivery Partner | Logistics personnel. Goals: Receive delivery assignments, update order status (Picked Up, Delivered), and view earnings. |
| Warehouse Staff  | Operational staff at dark stores. Goals: Update stock levels, mark items damaged/expired, perform stock audits.          |

| Role               | Description & Primary Goal                                                                                        |
|--------------------|-------------------------------------------------------------------------------------------------------------------|
| Operations Manager | Regional supervisors. Goals: Analyze demand trends, monitor warehouse performance, manage supplier relationships. |
| Supplier           | Vendors providing goods. Goals: Supply stock to warehouses (tracked via entry logs).                              |
| Admin              | System administrator. Goals: Manage master data (Product Catalog, Users, Warehouses), system configuration.       |

Each role maps to controlled database access privileges enforced at the application level.

## 5. In-Scope vs Out-of-Scope

### In-Scope

The following functionalities are within the scope of this database system:

- Complete Order Management Lifecycle (Order creation, processing, fulfillment, and completion)
- Warehouse and Batch-level Inventory Management
- Supplier Replenishment and Stock Entry Tracking
- Stock Adjustments due to damage, expiry, or loss with audit logs
- Basic Delivery Partner Assignment and Order Status Tracking
- Role-based Reporting and Analytics Queries
- Enforcement of data integrity using transactions, constraints, and ACID properties

### Out-of-Scope

The following features are intentionally excluded from the scope of this project:

- Real-time GPS tracking and map-based navigation
- Payment Gateway Integration (payments are simulated only)
- Advanced route optimization and driver routing algorithms
- Mobile or Web Application User Interface design
- Push notification and messaging systems
- Multi-city or multi-timezone operational support

## 6. Functional Requirements

The system must support the following core functions:

- **FR-1 User Management:** Registration and authentication for Customers, Staff, and Partners.
- **FR-2 Catalog Management:** Admin can create Products and Categories. Products are global; inventory is local to warehouses.

- **FR-3 Warehouse Selection:** System identifies the serviceable warehouse based on customer location (simulated via mapping).
- **FR-4 Batch Inventory:** Track stock by batches (Manufacturing Date, Expiry Date) to enforce First Expiry, First Out (FEFO)
- **FR-5 Order Placement:** Atomic transaction creating an Order, reserving stock from specific batches, and recording status.
- **FR-6 Stock Reservation:** Temporarily hold stock during checkout. If payment fails, stock is released.
- **FR-7 Delivery Management:** Assign a Delivery Partner to a confirmed order. Track status changes: Assigned -> Out for Delivery -> Delivered
- **FR-8 Replenishment:** Record incoming stock from Suppliers into specific Warehouses (increases inventory).
- **FR-9 Stock Adjustments:** Allow Warehouse Staff to reduce stock due to damage, theft, or expiry (audit log required).
- **FR-10 Reporting:** Generate sales, inventory, and performance reports for Managers.

## 7. Supported Queries / Reports

---

The database will support the following analytical queries, grouped by user role:

### *For Operations Managers:*

- **Daily Sales Report:** Total revenue and order count per warehouse for a given date range.
- **Top Selling Products:** List of top 10 products by volume in a specific region.
- **Low Stock Alert:** List products in a warehouse where quantity < threshold.
- **Expiry Report:** List batches expiring within the next 3 days.
- **Supplier Performance:** Total value of goods supplied by each vendor this month.

### *For Warehouse Staff:*

- **Pick List:** Items to be packed for all currently "Processing" orders.
- **Stock Ledger:** History of all stock movements (In/Out) for a specific product ID.

### *For Customers:*

- **Order History:** List past orders with status, total amount, and date.
- **Product Availability:** Check if Product X is available in the serving warehouse.

### *For Admin:*

- **User Activity:** Count of active customers registered in the last 30 days.
- **Delivery Performance:** Average delivery time per partner.

## 8. High-Level Data Requirements

---

### *Core Entities*

Product, Category, Warehouse, Supplier, Customer, Order, Order\_Item, Batch\_Inventory, Payment, Delivery\_Partner, Stock

## ***Key Relationships***

- **Warehouse ↔ Product:** Many-to-Many (Resolved via Inventory table).
- **Order ↔ Customer:** One-to-Many (One customer places many orders).
- **Order ↔ Warehouse:** Many-to-One (An order is fulfilled by one warehouse).
- **Order ↔ Delivery Partner:** Many-to-One.
- **Batch\_Inventory ↔ Product:** Many-to-One (One product has multiple expiry batches).

## ***Integrity Constraints & Business Rules***

- **Stock Non-Negativity:** Inventory quantity cannot be  $< 0$ .
- **Expiry Constraint:** System must not allow sale of batches where Expiry Date  $<$  Current Date.
- **Order Total Integrity:** Sum of (Item Price \* Quantity) must equal Order Total.
- **Orphan Data:** Orders cannot exist without a valid Customer ID.

## **9. Non-Functional Requirements**

---

- **Consistency (ACID):** All financial and inventory updates must occur within database transactions. A failure in payment must rollback stock reservation.
- **Performance:** Queries for "Product Availability" must execute in under 200ms to support instant browsing.
- **Security:** Passwords must be hashed. Users can only access data relevant to their role (Row Level Security logic at application level).
- **Auditability:** Every change in stock (addition, deduction, correction) must be logged in the *Stock\_Ledger* table with timestamp and user ID.
- **Scalability:** Database schema should support vertical partitioning if order volume grows significantly.

## **10. Assumptions & Constraints**

---

- **Database Technology:** MySQL (Relational DBMS).
- **Frontend Interface:** Web Application (Browser-based).
- **Single Region:** The prototype assumes operations within a single city/timezone for simplicity.
- **Currency:** All monetary values are in INR.
- **Delivery Assignment:** Currently manual or random assignment logic; sophisticated geospatial routing is out of scope.